TD / TP 5

Namespaces et cgroups

Attention aux copiés-collés des commandes sur un terminal. Certains caractères word peuvent ne pas correspondre aux caractères ASCII attendu par le terminal (les « , les -, etc.).

Exercice 1 : namespace pid et mount

Namespace id

1. Découvrez les namespace en cours d'utilisation. Tapez la commande `lsns` dans une console. Tapez la commande permettant d'afficher les namespace de votre bash (`lsns -p <pid>`). Vous pouvez récupérer le pid de votre shell avec la variable $$ ou la commande `ps`. A quelle namespace votre bash appartient il ?
2. Listez les namespace directement du système de fichiers (`/proc/<pid de votre shell>/ns/`). Vérifiez que vous retrouvez bien les mêmes namespace que précédemment. Quel est le type de ces fichiers ? Vers quoi pointent ils ?

Namespace pid

3. Lancez un nouveau shell qui aura les mêmes namespaces que son père sauf pour « pid » (l'option --fork indique que le programme sera exécuté dans un processus fils – c'est unshare qui se fork).

```
sudo unshare  --fork --pid bash
```

4. Indiquez le pid du processus dans le nouveau bash (echo $$) et dans le host (ouvrez une autre console pour ce faire et utilisez la commande ps a). Nous appelerons cette deuxième console la console host dans la suite.
5. Listez tous les processus dans les 2 consoles (ps aux | grep bash). La commande ps utilisant la liste des processus se trouvant dans le répertoire /proc, les deux processus doivent afficher la même chose.
6. Les namespace sont ils les mêmes (commande `lsns -p <pid>` ou directement dans `/proc/<pid>/ns/`) ? Il faut regarder les numéros d'inode pour vérifier quelles sont les namespace commun. Autre méthode, vous pouvez aussi lire les liens symboliques dans chacune des deux consoles (readlink /proc/$$/ns/*). Quittez (exit).

Namespace mount

7. Lancez un nouveau shell qui aura les mêmes namespace que son père sauf le namespace « mount »

```
sudo unshare  --fork --mount bash
```

8. Insérez une clé ou un périphérique sur votre vm. Démontez là et montez à la main dans votre bash (celui de namespace mount différent). Pour ce faire créer un dossier (« toto » dans l'exemple ci-dessous »).

```
sudo mount  /dev/sdb1   toto
```

Si /dev/sdb1 ne fonctionne pas, la commande fdisk –l vous permettra de connaître le fichier /dev correspondant à votre clé.

9. Faites un ls du dossier « toto » dans le nouveau bash de namespace différent et dans la console host.
10. Démontez la clé (`umount toto`). Il faut le faire dans la bonne console !
11. Cela fonctionne aussi pour des dossiers ordinaires. Identifiez un dossier avec du contenu (disons « folder »). Et dans le bash de namespace différent tapez la commande :
```
sudo mount --bind folder toto
```
12. Testez le contenu du dossier toto dans les deux bash/consoles.
13. Créez un dossier etc dans votre répertoire courant (pas à la racine bien sûr). Mettez un fichier de configuration bidon dedans. Montez ce dossier comme le fichier etc de la racine :
```
mount --bind /etc /chemin/etc
```
14. Afin d'isoler la vue des processus, il est possible de combiner le namespace « pid » avec le montage d'un /proc qui lui est propre. Tapez la commande
```
sudo unshare --pid  --fork --mount-proc bash
```
15. Testez ps dans les deux consoles (celle qui est « unshare » et la console host).
16. Listez les processus en cours d'exécution en parcourant le dossier /proc. Comme indiqué plus haut vous avez un dossier par processus (le nom du dossier est le pid du processus). Faites ce « ls » dans les deux consoles.
17. Comparez les namespace des deux bash (unshared et classique). Cela se fait dans la console host : identifiez les pid des deux bash, consultez le dossier `/proc/<pid>/ns` (commande `ls –l` pour voir les identifiants des inodes). Vous devez observer qu'en plus du namespace pid et pid_children qui est différent, le montage d'un /proc différent à nécessité la création d'un namespace mount spécifique pour le nouveau bash.
18. Quittez.

<u>Exercice 2 : namespace UTS et user</u>

Garder deux consoles, une dans laquelle vous changerez les namespace et leur propriété (la console « unshare »), et une qui ne sera pas modifié que nous appellerons la console host.

1. Créer un nouveau bash avec un nouveau namespace uts
```
sudo unshare --fork --uts bash
```

Nous allons changer le hostname de deux manière :

2. Dans la console unshare tapez la commande `hostname`. Le hostname courant est affiché. C'est le même dans la console host. Changez le (dans la console unshare) : hostname toto. Affichez le dans les deux consoles (commande hostname).
3. Changez le hostname dans la console host. Celui de la console unshare a-t-il été changé ?
4. Notez le pid du bash de la console unshare (commande ps). Dans la console host, tapez la commande
```
sudo nsenter --target <pid> --uts
```

5. Vous avez un bash pour lequel toutes les commandes tapées auront effet sur le namespace uts du processus de pid <pid>. Changez le hostname dans la console host (hostname taratata) puis quittez nsenter. Affichez les hostname dans les deux consoles. Vérifiez bien que la commande nsenter/hostname a été appliqué à la console unshare uniquement. Quittez.
6. Créer un nouveau bash avec un nouveau namespace user avec l'option « root mapping ». Votre utilisateur sera donc l'utilisateur devops à l'extérieur du namespace (rien de changer à ce niveau là) et root à l'intérieur du nouveau namespace. Attention, la commande qui suit ne doit pas être tapé en mode sudo.

```
unshare  --user -r  bash
```

7. Affichez le mapping des utilisateurs pour ce namespace

```
cat  /proc/<pid>/uid_map
```

   où <pid> est le pid de votre bash (vous pouvez faire un ps ou le remplacer par $$ dans la commande du dessus).
8. Changez le hostname et vérifiez sa prise en compte (hostname toto puis hostname si la première commande a fonctionné). Quittez.
9. Vous allez maintenant créer plusieurs namespace (différents de celui du père) pour lesquels vous aurez les droits root : namespace user et uts. Ne le faites pas en tant que sudo (notez que sans le namespace user vous n'auriez pas les droits).

```
unshare --user –r --uts --fork bash
```

10. Changez le hostname et vérifiez sa prise en compte (hostname toto puis hostname si la première commande a fonctionné).

<u>Exercice 3 : cgroups</u>

Vous pouvez soit utiliser les commandes de la librairie libcgroup (voir les man) ou directement les commandes manuelles (modification des fichiers/dossiers dans /sys/fs/cgroup/).

1. Créer un cgroup memory et cgroup cpu. Pour cela il faut créer un dossier (nom au choix) dans les dossiers /sys/fs/cgroup/cpu/ et /sys/fs/cgroup/memory/.
2. Vous pouvez lancez quelques processus (script qui boucle sur sleep par exemple). Rajoutez les dans les cgroup que vous avez créé. Il vaut faut rajouter les pid de ces processus dans les fichiers cgroup.procs dans les dossiers que vous avez créés précédemment.
3. Limitez le CPU de ces processus. Il faut fixer une valeur entre 1 et 1024 (100%) dans le fichier cpu.shares (toujours dans les dossiers que vous avez créé).
4. Associez aussi votre propre shell à ce cgroup (cpu) et lancez quelques uns de vos scripts en tâche de fond. Affichez le contenu du fichier cgroup.procs. Vous devez y voir votre shell et ses fils (ainsi que les processus précédents et leurs fils si ils en ont).
5. Limitez la mémoire de ces processus. Il faut fixer une valeur en octets dans le fichier memory.limit_in_bytes. Testez avec une grande valeur (plusieurs dizaines de ko et une petite valeur). Pour la petite valeur vous constaterez que ces processus sont tués lorsqu'ils dépassent ces limites.

L'exercice 4 doit être rendu. Vous avez le choix entre les deux exercices 4 ci-dessous.

Exercice 4 : créer son conteneur avec un script bash (difficile)

Vous devez écrire un script qui va isoler un environnement d'exécution. Le processus principal/initial sera un bash. Votre script doit être plug&play et aucune tâche ne doit être effectuée en dehors du script. Voici ce qui est demandé :

- Le processus initial sera un bash
- Le hostname devra être différent
- Il devra aussi être isolé du point de vue du réseau et des processus (namespaces net et pid)
- Il devra être chrooté mais les dossiers /media et /tmp (pour pouvoir accéder aux périphériques type usb et aux fichiers temporaires) doivent correspondre aux mêmes que le SE
- Les commandes ls, touch, cat doivent être implémentées à minima
- Il faudra rajouter une commande « myCommand » qui correspond à une commande propre et qui pourra tourner en tâche de fond (l'idée étant d'observer la filiation au sein de votre environnement).
- Les ressources mémoires et CPU doivent être limitées (à votre guise).

Le script doit être commenté et facilement lisible. Comment vérifier que cela marche :

- Affichez les pid (ils doivent être numéroté 1, 2, etc.) au sein de votre environnement
- Il ne doit pas pouvoir accéder au système de fichiers (sauf sa propre branche) excepté les dossiers /media et /tmp. Vous pouvez brancher une clé usb pour vérifier.
- Il ne doit pas pouvoir accéder au réseau (il y a des choses à faire en plus pour que ce soit le cas).
- Le hostname doit être différent.
- Seuls les quelques commandes que vous avez intégrées doivent fonctionner.
- Vous pouvez aussi vérifier que les limitations mémoires sont bien pris en compte en testant avec une petite allocation par exemple.

Exercice 4 : un conteneur simple en C

Il vous est demandé d'écrire un programme C simple qui créer un conteneur. Vous pourrez partir du squelette fournit par votre enseignant. Ce qui est demandé :

1. Avant le fork, le processus doit appeler unshare afin de créer un nouveau namespace processus
2. Le processus se fork (déjà fait)
3. Le processus fils doit appeler unshare de manière à créer les autres namespace (qui lui seront propre). Il faudra au moins celui du système de fichiers (namespace fs/mount).
4. Le processus fils effectue un chroot. Au préalable, vous aurez créé les dossiers bin, lib, lib64, tmp, proc et media et copié quelques commandes (à minima bash, ps, kill et ls) dans bin et les librairies correspondantes. Rappel : commande ldd pour trouver les librairies correspondants à vos binaires.
5. Le processus fils recouvre son code avec celui du bash (execlp).

Remarque : il est aussi possible d'utiliser l'appel système clone avec des options qui permettent de cloner le processus courant et de l'associer à des namespace différents.

Exercice 5 (facultatif) :

Cet exercice va essentiellement utiliser la commande ip combiné à l'option netns qui permet de manipuler directement les namespace network. La commande unshare aurait pu aussi être utilisée pour créer ces namespace networks (mais moins pratique ici).

Préambule :

1. Créer un nouveau namespace network que l'on appellera newNet :
```
ip netns add newNet
ip netns list //Vérifie la liste des namespace net
```
2. Affichez votre adresse ip dans le nouveau namespace pour l'interface lo (loopback) et si il n'y en a pas assigné l'adresse 127.0.0.1 puis activez l'interface :
```
sudo ip netns exec newNet ip addr add 127.0.0.1/32 dev lo
sudo ip -n newNet link set lo up
```
3. Lancer nginx dans le nouveau namespace (pensez bien à arrêter apache2 et nginx avant):
```
ip netns exec newNet /usr/sbin/nginx
```
Ne pas lancez nginx (ou apache2) à partir des scripts (service ou etc/init.d) car c'est lancé avec systemctl dans le namespace root.
4. Lancez votre navigateur et affichez la page par défaut (http://127.0.0.1). Cela marche-t-il ?
5. Téléchargez la page web du namespace avec wget (cela ne fonctionnera pas avec firefox car il refusera d'être lancé en root) :
```
sudo ip netns exec newNet wget 127.0.0.1
```
Vous pouvez télécharger la page web. Il apparaît donc que les deux réseaux sont bien isolés : celui du namespace root et celui de votre nouveau namespace. Les services qui sont donc lancés d'un autre namespace network ne sont pas visible de la part des autres namespace net. Il est aussi possible d'avoir plusieurs services identiques sur des namespace network différents.

1ère partie : communication entre un namespace network et le namespace root.

1. Créer un nouveau namespace toto :
```
ip netns add toto
```
2. Créez deux interfaces virtuelles et connectez les:
`ip link add veth-root type veth peer name veth-toto` //Créer l'interface virtuelle et son extrémité (créer aussi l'autre interface veth-toto)
3. Effectuez un « ip link ». Vous voyez les deux nouvelles interfaces nouvellement créées.
4. Associez l'une des deux interfaces dans le namespace toto. L'autre restera dans le namespace root.
`ip link set veth-toto netns toto` //associe l'interface veth-tot au namespace toto
5. Effectuez un « ip link », l'interface veth-toto a disparu (car rattachée à l'autre namespace).
6. Associez une adresse ip à chacune des interfaces :
```
ip addr add 192.168.0.2/24 dev veth-root
ip -n toto addr add 192.168.0.1/24 dev veth-toto
```
7. On active les interfaces
```
ip link set veth-root up
```

```
ip -n toto link set veth-toto up
```

8. Les deux interfaces peuvent se pinguer:

   `ping 192.168.0.1` //le namespace root ping le namepsace toto

   `ip netns exec toto ping 192.168.0.2` //La commande ping est effectué dans le namespace toto et ping le namespace root

9. Pour la connection à Internet cela se fait en plusieurs étapes (voir exercice plus bas). Cependant, si vous aviez mis des adresses IP publiques valides à vos deux interfaces veth il suffirait de rajouter une route par défaut :

   ```
   ip netns exec toto ip route add default via <@IP de veth-root>
   ```

   A minima, vous pouvez toujours tester l'accès à l'interface physique :

   ```
   ip netns exec toto ip route add default via 192.168.0.2
   ```

   ```
   ip netns exec toto ping <@IP de votre interface physique>
   ```

<u>2ème partie :</u> Connection entre deux namespace.

10. Supprimez l'ancien namespace toto (ip netns del toto). Créez deux nouveau namespace networks appelé toto et titi.

    ```
    ip netns add toto
    ip netns add titi
    ```

11. Visualisez vos interfaces dans ces deux namespaces :

    `ip netns exec toto ip link` //effectue la commande ip link dans le namespace toto

    `ip netns exec titi ip link`

12. Les interfaces, exceptés l'interface de loopback, ont disparu. Les processus de ce namespace sont isolés (du point de vue du réseau). Une commande alternative est : ip –n titi link

13. Vous allez créer une interface Ethernet virtuelle sur chacun des deux namespace. Cela se fait en deux étapes :

    `ip link add veth-titi type veth peer name veth-toto` //Créer l'interface virtuelle et de son extrémité (créer aussi l'autre interface veth-toto)

    `ip link set veth-titi netns titi` //Place l'interface veth-titi dans le namespace titi

    `ip ink set veth-toto netns toto` //Place l'interface veth-toto dans le namespace toto

    ou en une seule commande :

    ```
    ip link add <p1-name> netns <p1-ns> type veth peer <p2-name> netns <p2-ns>
    ```

14. Les deux namespaces ont maintenant chacun une interface qui sont connectés entre elles.

15. On leur assigne des adresses ip sur le même sous réseau :

    ```
    ip -n toto addr add 192.168.0.1/24 dev veth-toto
    ip -n titi addr add 192.168.0.2/24 dev veth-titi
    ```

16. On active les interfaces

    ```
    ip -n titi link set veth-titi up
    ```

```
ip –n toto link set veth-toto up
```

17. Vérifiez que cela a bien fonctionné avec la commande ip link (dans le bon namespace !). Est-ce que ces interfaces sont visibles dans le namespace root ?
18. Faites un ping entre les interfaces (la commande ip netns exec xxx exécute une commande dans le namespace network xxx):

```
ip netns exec toto ping 192.168.0.2
```

19. Vos deux namespace peuvent communiquer!

<u>3ème partie :</u> deux namespace communiquent ensemble et avec l'extérieur.

On souhaite communiquer maintenant avec l'extérieur. Nous allons créer un bridge (virtuel) qui sert de pont/commutateur entre les namespace. Il y a deux types de bridge : les bridge Linux utilisés ci-dessous, et les bridge openvswitch.

1. On créer le bridge Linux.
   `ip link add myLinuxBridge type Bridge` //Création d'un bridge Linux (dans le namespace root)
   `ip link set dev myLinuxBridge up` //Activation de l'interface

2. On créer des liens de chaque interface des namespace vers le bridge. Cela se fait en deux étapes, création d'un lien partant de l'interface virtuel vers une extrémité. Puis rattachement de cette extrémité sur le bridge.
   `ip link add veth-toto type veth peer name veth-toto-bridge` //on le retape car l'extrémité à changer
   `ip link set veth-toto netns toto` //interface rattachée au namespace toto
   L'extrémité du lien est associée au bridge :
   `ip link set veth-toto-br master myLinuxBridge`
3. On assigne les addresses ip aux interfaces et on active les interfaces (voir commandes ci-dessus). Vous assignerez les adresses 192.168.0.1/24 à l'interface du namespace toto, 192.168.0.2/24 au namespace titi et 192.168.0.3/24 au bridge (myLinuxBridge).
4. Les deux interfaces des namespace toto et titi doivent pouvoir se pinguer.
5. Afficher les tables de routage :
   `ip netns exec toto ip route`
6. Pour sortir du sous réseau 192.168.0.0/24, il faut rajouter une route par défaut dans chacun des namespace :
   `ip netns exec toto ip route add default via 192.168.0.3`
7. Comme dans la 1ère partie. Si les adresses avaient été publiques et valides, cela aurait suffi pour se connecter à l'Internet. Comme les adresses sont privées, il faut faire du NAT. On pourra donc taper la commande suivante si l'adresse à utiliser pour sortir du réseau doit être celle de l'interface du PC :

```
iptable –t –nat –A POSTROUTING –s @reseauDuNamespace/mask
-j MASQUERADE
```

8. et faire du PAT éventuellement si on veut être accessile de l'éxterieur.

```
ip –t nat –A PREROUTING –d-port 80 –to-destination
@IPinterne –j DNAT
```

Notez qu'il est possible de créer n'importe quelle topologie virtuelle entre les namespace et le namespace root avec les interfaces virtuelles, les bridges, et des routes. Il est donc possible par exemple de créer des namespaces qui auront une interface vers un autre namespace qui filtrera les paquets, qui offrira d'autres services réseau, ou de translation d'adresses, etc.